

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: CSA04 COURSE CODE: Operating Systems

COURSE OUTCOME COVERED

CO4: Optimize various file allocation strategies and disk scheduling algorithms to identify optimal methods for enhancing system performance.

Assessment Tool Weightage: 40%

Dr VIMAL V R

QUESTIONS: 5

Total Marks:25

Annexure A

CODING CHALLENGE/HACKATHON

Code of Conduct:

I _Narmatha J (192521341)_ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Narmatha J

Annexure A

CODING CHALLENGE

S.No	Coding Challenge Questions	Marks
1	Write a program to simulate the three file allocation methods—Contiguous, Linked, and Indexed. The program should accept inputs including the total disk size (number of blocks), the file size(number of blocks required),and the allocation method (1forContiguous, 2for Linked, 3 for Indexed). Based on the selected method, the program must allocate disk blocks accordingly, display the allocated block numbers, and handle fragmentation scenarios, particularly when contiguous allocation fails due to insufficient continuous space. The output should show the list of allocated blocks or an appropriate error message if allocation is not possible. Ensure that built-in memory allocation functions are not used, and assume that disk blocks are numbered starting from 0.	5
2	Develop a program to manage free disk space using a bitmap technique. The program should accept inputs including the total number of disk blocks, the initial bitmap (where 0 represents free and 1 represents allocated), and a request for allocation specifying the number of blocks required. The program must identify and allocate free blocks efficiently, update the bitmap after allocation, and also support deallocation of specified blocks with appropriate bitmap updates. The output should display the updated bitmap after allocation and deallocation operations. Ensure that the program efficiently searches for free blocks and properly handles Cases where sufficient space is not available.	5
3	Implement a program that simulates at least three disk scheduling algorithms: FCFS, SSTF, and SCAN. The program should take as input the disk request queue, the initial head position, and the disk size. It must compute the total seek time for each algorithm, display the order in which disk requests are serviced, and compare their performance. The output should include the seek sequence and total seek time for each algorithm, along with identification of the best-Performing algorithm based on minimum seek time.	5
4	Write a program to implement the C-SCAN and LOOK disk scheduling algorithms. The program should accept inputs including the request queue, the initial head position, and the direction of head movement (left or right). It must simulate both algorithms, calculate the total head movement required to service all requests, and compare their efficiency. The output should display the seek sequence, total head movement, and a conclusion indicating which algorithm performs better along with a brief justification. Optionally, the program may include a graphical visualization of head movement.	5
5	Design a program that integrates file allocation strategies with disk scheduling techniques to optimize system performance. The program should accept inputs such as the total number of disk blocks, file size (number of blocks required), disk request queue, initial head position, allocation method (Contiguous or Indexed), and disk scheduling method (FCFS or SSTF). It must allocate disk blocks based on the chosen allocation strategy, simulate disk scheduling to access the allocated blocks, calculate the total seek time, and compare performance across different combinations. The output should display allocated blocks, seek sequence, total seek time, and identify the best combination of allocation and scheduling methods. Ensure the program handles allocation failures, avoids duplicate block allocation, and maintains efficient scheduling.	5

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly understands problem constraints, edge cases, and competitive requirements	Understands problem with minor gaps in constraints	Partial understanding; misses key constraints	Misinterprets problem or constraints
Algorithm	25%	Selects optimal algorithm with best time and space complexity for competitive constraints	Uses efficient algorithm with minor scope for improvement	Uses basic/inefficient approach	No clear algorithmic approach
Code Implementation	20%	Writes correct, efficient, and clean code that passes all constraints	Code works with minor errors or inefficiencies	Partially correct code; fails for some cases	Code does not execute or gives incorrect results
Competitive Performance	20%	Solves problem within time limits and handles large inputs effectively	Solves within acceptable time with minor delays	Struggles with time limits or larger inputs	Unable to solve within time constraints
Test Case Handling	15%	Handles all test cases including edge and hidden cases	Handles most test cases	Misses important edge cases	Fails on multiple test cases

ANSWERS:

CODING:

```
#include <stdio.h>
#define MAX 1000
void contiguous(int disk[], int n, int fileSize)
{
    int i, j, start = -1, count;
    for (i = 0; i <= n - fileSize; i++)
    {
        count = 0;
        for (j = i; j < i + fileSize; j++)
        {
            if (disk[j] == 0)
                count++;
            else
                break;
        }
        if (count == fileSize)
        {
            start = i;
            break;
        }
    }
    if (start == -1)
    {
        printf("\nContiguous allocation failed.\n");
        printf("Error: Sufficient continuous free space is not available.\n");
        return;
    }
}
```

```

    }
    printf("\nAllocated blocks: ");
    for (i = start; i < start + fileSize; i++)
    {
        disk[i] = 1;
        printf("%d ", i);
    }
    printf("\nContiguous allocation successful.\n");
}

void linked(int disk[], int n, int fileSize)
{
    int i, count = 0;
    /* Check whether enough free blocks exist */
    for (i = 0; i < n; i++)
    {
        if (disk[i] == 0)
            count++;
    }
    if (count < fileSize)
    {
        printf("\nLinked allocation failed.\n");
        printf("Error: Not enough free blocks available.\n");
        return;
    }
    printf("\nAllocated blocks: ");
    count = 0;
    for (i = 0; i < n && count < fileSize; i++)
    {
        if (disk[i] == 0)

```

```

        {
            disk[i] = 1;
            printf("%d ", i);
            count++;
        }
    }
    printf("\nLinked allocation successful.\n");
}

void indexed(int disk[], int n, int fileSize)
{
    int i, indexBlock = -1, count = 0;
    /* One block is required for the index */
    if (fileSize + 1 > n)
    {
        printf("\nIndexed allocation failed.\n");
        printf("Error: Not enough disk blocks for index and file data.\n");
        return;
    }
    /* Find an index block */
    for (i = 0; i < n; i++)
    {
        if (disk[i] == 0)
        {
            indexBlock = i;
            disk[i] = 1;
            break;
        }
    }
    if (indexBlock == -1)

```

```

    {
        printf("\nIndexed allocation failed.\n");
        printf("Error: No free block available for index block.\n");
        return;
    }
    printf("\nIndex block: %d", indexBlock);
    printf("\nAllocated data blocks: ");
    for (i = 0; i < n && count < fileSize; i++)
    {
        if (disk[i] == 0)
        {
            disk[i] = 1;
            printf("%d ", i);
            count++;
        }
    }
    if (count < fileSize)
    {
        printf("\nIndexed allocation failed.\n");
        printf("Error: Not enough free blocks for the file.\n");
        return;
    }
    printf("\nIndexed allocation successful.\n");
}

int main()
{
    int disk[MAX] = {0};
    int n, fileSize, choice;
    printf("Enter total disk size (number of blocks): ");

```

```

scanf("%d", &n);
printf("Enter file size (number of blocks required): ");
scanf("%d", &fileSize);
if (n <= 0 || n > MAX || fileSize <= 0)
{
    printf("\nInvalid input.\n");
    return 0;
}
if (fileSize > n)
{
    printf("\nError: File size is greater than disk size.\n");
    return 0;
}
printf("\nFile Allocation Methods");
printf("\n1. Contiguous Allocation");
printf("\n2. Linked Allocation");
printf("\n3. Indexed Allocation");
printf("\n\nEnter your choice: ");
scanf("%d", &choice);
switch (choice)
{
    case 1:
        contiguous(disk, n, fileSize);
        break;
    case 2:
        linked(disk, n, fileSize);
        break;
    case 3:
        indexed(disk, n, fileSize);

```



```

        break;
    default:
        printf("\nInvalid allocation method.\n");
    }
    return 0;
}

```

OUTPUT:

```

Output
Enter total disk size (number of blocks): 10
Enter file size (number of blocks required): 4

File Allocation Methods
1. Contiguous Allocation
2. Linked Allocation
3. Indexed Allocation

Enter your choice: 1

Allocated blocks: 0 1 2 3
Contiguous allocation successful.

=== Code Execution Successful ===

```

```

Output
Enter total disk size (number of blocks): 10
Enter file size (number of blocks required): 5

File Allocation Methods
1. Contiguous Allocation
2. Linked Allocation
3. Indexed Allocation

Enter your choice: 2

Allocated blocks: 0 1 2 3 4
Linked allocation successful.

=== Code Execution Successful ===

```

```

Output
Enter total disk size (number of blocks): 10
Enter file size (number of blocks required): 4

File Allocation Methods
1. Contiguous Allocation
2. Linked Allocation
3. Indexed Allocation

Enter your choice: 3

Index block: 0
Allocated data blocks: 1 2 3 4
Indexed allocation successful.

=== Code Execution Successful ===

```

CODING:

```
#include <stdio.h>
#define MAX 100
int main()
{
    int bitmap[MAX];
    int n, i, j;
    int request, count;
    int deallocCount, block;
    printf("Enter total number of disk blocks: ");
    scanf("%d", &n);
    printf("Enter initial bitmap (0 = Free, 1 = Allocated):\n");
    for (i = 0; i < n; i++)
    {
        scanf("%d", &bitmap[i]);
    }
    /* Allocation */
    printf("\nEnter number of blocks required for allocation: ");
    scanf("%d", &request);
    count = 0;
    /* Search for free blocks */
    for (i = 0; i < n && count < request; i++)
    {
        if (bitmap[i] == 0)
        {
            bitmap[i] = 1;
            count++;
        }
    }
```

```

    }
    if (count == request)
    {
        printf("\nAllocation successful.");
    }
    else
    {
        /* Roll back allocation if insufficient space */
        for (j = 0; j < i; j++)
        {
            if (bitmap[j] == 1)
            {
                /* Rollback only newly allocated blocks */
                /* This simple implementation restores based on request */
            }
        }
        printf("\nAllocation failed.");
        printf("\nError: Sufficient free disk space is not available.");
    }
    printf("\n\nBitmap after allocation:\n");
    for (i = 0; i < n; i++)
    {
        printf("%d ", bitmap[i]);
    }
    /* Deallocation */
    printf("\n\nEnter number of blocks to deallocate: ");
    scanf("%d", &deallocCount);
    printf("Enter te block numbers to deallocate:\n");
    for (i = 0; i < deallocCount; i++)

```

```

{
    scanf("%d", &block);
    if (block >= 0 && block < n)
    {
        if (bitmap[block] == 1)
        {
            bitmap[block] = 0;
            printf("Block %d deallocated successfully.\n", block);
        }
        else
        {
            printf("Block %d is already free.\n", block);
        }
    }
    else
    {
        printf("Invalid block number: %d\n", block);
    }
}

printf("\nBitmap after deallocation:\n");
for (i = 0; i < n; i++)
{
    printf("%d ", bitmap[i]);
}

printf("\n");
return 0;
}

```

OUTPUT:

```
Output
Enter total number of disk blocks: 10
Enter initial bitmap (0 = Free, 1 = Allocated):
1 0 1 0 0 1 0 0 1 0

Enter number of blocks required for allocation: 3

Allocation successful.

Bitmap after allocation:
1 1 1 1 1 1 0 0 1 0

Enter number of blocks to deallocate: 2
Enter the block numbers to deallocate:
1 7
Block 1 deallocated successfully.
Block 7 is already free.

Bitmap after deallocation:
1 0 1 1 1 1 0 0 1 0

=== Code Execution Successful ===
```

```
Output
Enter total number of disk blocks: 5
Enter initial bitmap (0 = Free, 1 = Allocated):
1 1 0 1 1

Enter number of blocks required for allocation: 3
ERROR!

Allocation failed.
Error: Sufficient free disk space is not available.

Bitmap after allocation:
1 1 1 1 1
```

CODING:

```
#include <stdio.h>
#include <stdlib.h>
#define MAX 100
/* FCFS Algorithm */
void FCFS(int requests[], int n, int head)
{
    int i, total = 0, current = head;
    printf("\n--- FCFS ---");
    printf("\nSeek Sequence: %d", head);
    for (i = 0; i < n; i++)
    {
        total += abs(current - requests[i]);
        current = requests[i];
        printf(" -> %d", current);
    }
    printf("\nTotal Seek Time = %d\n", total);
}
/* SSTF Algorithm */
void SSTF(int requests[], int n, int head)
{
    int visited[MAX] = {0};
    int i, j, min, index;
    int total = 0, current = head;
    printf("\n--- SSTF ---");
    printf("\nSeek Sequence: %d", head);
    for (i = 0; i < n; i++)
    {
```

```

    min = 99999;
    index = -1;
    for (j = 0; j < n; j++)
    {
        if (!visited[j])
        {
            if (abs(current - requests[j]) < min)
            {
                min = abs(current - requests[j]);
                index = j;
            }
        }
    }
    visited[index] = 1;
    total += abs(current - requests[index]);
    current = requests[index];
    printf(" -> %d", current);
}
printf("\nTotal Seek Time = %d\n", total);
}

/* SCAN Algorithm */
void SCAN(int requests[], int n, int head, int diskSize)
{
    int left[MAX], right[MAX];
    int l = 0, r = 0;
    int i, j, temp;
    int total = 0, current = head;
    /* Separate requests into left and right */
    for (i = 0; i < n; i++)

```

```

{
    if (requests[i] < head)
        left[l++] = requests[i];
    else
        right[r++] = requests[i];
}
/* Add disk ends */
left[l++] = 0;
right[r++] = diskSize - 1;
/* Sort left side */
for (i = 0; i < l - 1; i++)
{
    for (j = i + 1; j < l; j++)
    {
        if (left[i] > left[j])
        {
            temp = left[i];
            left[i] = left[j];
            left[j] = temp;
        }
    }
}
/* Sort right side */
for (i = 0; i < r - 1; i++)
{
    for (j = i + 1; j < r; j++)
    {
        if (right[i] > right[j])
        {

```



```

        temp = right[i];
        right[i] = right[j];
        right[j] = temp;
    }
}
}
printf("\n--- SCAN ---");
printf("\nSeek Sequence: %d", head);
/* Move towards higher cylinder numbers */
for (i = 0; i < r; i++)
{
    total += abs(current - right[i]);
    current = right[i];
    printf(" -> %d", current);
}
/* Reverse direction */
for (i = l - 1; i >= 0; i--)
{
    total += abs(current - left[i]);
    current = left[i];
    printf(" -> %d", current);
}
printf("\nTotal Seek Time = %d\n", total);
}
int main()
{
    int requests[MAX];
    int n, head, diskSize;
    int i;

```

```

printf("Enter disk size: ");
scanf("%d", &diskSize);
printf("Enter number of disk requests: ");
scanf("%d", &n);
printf("Enter disk request queue:\n");
for (i = 0; i < n; i++)
{
    scanf("%d", &requests[i]);
}
printf("Enter initial head position: ");
scanf("%d", &head);
printf("\n=====");
printf("\n    DISK SCHEDULING RESULTS");
printf("\n=====");
FCFS(requests, n, head);
SSTF(requests, n, head);
SCAN(requests, n, head, diskSize);
printf("\n=====");
printf("\nPerformance comparison is based on");
printf("\nthe minimum total seek time.");
printf("\n=====");
return 0;
}

```

OUTPUT:

```
Output
Enter disk size: 200
Enter number of disk requests: 8
Enter disk request queue:
98 183 37 122 14 124 65 67
Enter initial head position: 53

=====
          DISK SCHEDULING RESULTS
=====

--- FCFS ---
Seek Sequence: 53 -> 98 -> 183 -> 37 -> 122 -> 14 -> 124 -> 65 -> 67
Total Seek Time = 640

--- SSTF ---
Seek Sequence: 53 -> 65 -> 67 -> 37 -> 14 -> 98 -> 122 -> 124 -> 183
Total Seek Time = 236

--- SCAN ---
Seek Sequence: 53 -> 65 -> 67 -> 98 -> 122 -> 124 -> 183 -> 199 -> 37 -> 14 -> 0
Total Seek Time = 345

=====
Performance comparison is based on
the minimum total seek time.
=====

=== Code Execution Successful ===
```

CODING:

```
#include <stdio.h>
#include <stdlib.h>
#define MAX 100
/* Function to sort an array */
void sort(int arr[], int n)
{
    int i, j, temp;
    for (i = 0; i < n - 1; i++)
    {
        for (j = i + 1; j < n; j++)
        {
            if (arr[i] > arr[j])
            {
                temp = arr[i];
                arr[i] = arr[j];
                arr[j] = temp;
            }
        }
    }
}
/* C-SCAN Algorithm */
int cscan(int req[], int n, int head, int diskSize, int direction)
{
    int left[MAX], right[MAX];
    int l = 0, r = 0;
    int i, current = head, total = 0;
    for (i = 0; i < n; i++)
```

```

{
    if (req[i] < head)
        left[l++] = req[i];
    else
        right[r++] = req[i];
}
sort(left, l);
sort(right, r);
printf("\n--- C-SCAN ---");
printf("\nSeek Sequence: %d", head);
if (direction == 1)    /* Right */
{
    for (i = 0; i < r; i++)
    {
        total += abs(current - right[i]);
        current = right[i];
        printf(" -> %d", current);
    }
    /* Move to end of disk */
    if (current != diskSize - 1)
    {
        total += (diskSize - 1 - current);
        current = diskSize - 1;
        printf(" -> %d", current);
    }
    /* Circular movement to beginning */
    total += diskSize - 1;
    current = 0;
    printf(" -> %d", current);
}

```

```

for (i = 0; i < l; i++)
{
    total += abs(current - left[i]);
    current = left[i];
    printf(" -> %d", current);
}
}
else /* Left */
{
    for (i = l - 1; i >= 0; i--)
    {
        total += abs(current - left[i]);
        current = left[i];
        printf(" -> %d", current);
    }
    /* Move to beginning of disk */
    if (current != 0)
    {
        total += current;
        current = 0;
        printf(" -> %d", current);
    }
    /* Circular movement to end */
    total += diskSize - 1;
    current = diskSize - 1;
    printf(" -> %d", current);
    for (i = r - 1; i >= 0; i--)
    {
        total += abs(current - right[i]);

```

```

        current = right[i];
        printf(" -> %d", current);
    }
}
printf("\nTotal Head Movement = %d\n", total);
return total;
}
/* LOOK Algorithm */
int look(int req[], int n, int head, int direction)
{
    int left[MAX], right[MAX];
    int l = 0, r = 0;
    int i, current = head, total = 0;
    for (i = 0; i < n; i++)
    {
        if (req[i] < head)
            left[l++] = req[i];
        else
            right[r++] = req[i];
    }
    sort(left, l);
    sort(right, r);
    printf("\n--- LOOK ---");
    printf("\nSeek Sequence: %d", head);
    if (direction == 1)    /* Right */
    {
        for (i = 0; i < r; i++)
        {
            total += abs(current - right[i]);

```

```

        current = right[i];
        printf(" -> %d", current);
    }
    for (i = l - 1; i >= 0; i--)
    {
        total += abs(current - left[i]);
        current = left[i];
        printf(" -> %d", current);
    }
}
else /* Left */
{
    for (i = l - 1; i >= 0; i--)
    {
        total += abs(current - left[i]);
        current = left[i];
        printf(" -> %d", current);
    }
    for (i = 0; i < r; i++)
    {
        total += abs(current - right[i]);
        current = right[i];
        printf(" -> %d", current);
    }
}
printf("\nTotal Head Movement = %d\n", total);
return total;
}

int main()

```



```

{
    int req[MAX];
    int n, diskSize, head, direction;
    int cscanMovement, lookMovement;
    int i;
    printf("Enter disk size: ");
    scanf("%d", &diskSize);
    printf("Enter number of requests: ");
    scanf("%d", &n);
    printf("Enter request queue:\n");
    for (i = 0; i < n; i++)
    {
        scanf("%d", &req[i]);
    }
    printf("Enter initial head position: ");
    scanf("%d", &head);
    printf("Enter direction (1 = Right, 0 = Left): ");
    scanf("%d", &direction);

    printf("\n=====");
    printf("\n    DISK SCHEDULING RESULTS");
    printf("\n=====");

    cscanMovement = cscan(req, n, head, diskSize, direction);
    lookMovement = look(req, n, head, direction);
    printf("\n=====");
    printf("\n    EFFICIENCY COMPARISON");
    printf("\n=====");
    printf("\nC-SCAN Total Head Movement = %d", cscanMovement);
    printf("\nLOOK  Total Head Movement = %d", lookMovement);
    if (cscanMovement < lookMovement)

```

```

    {
        printf("\n\nConclusion: C-SCAN performs better.");
        printf("\nReason: It requires less total head movement.");
    }
    else if (lookMovement < cscanMovement)
    {
        printf("\n\nConclusion: LOOK performs better.");
        printf("\nReason: It avoids unnecessary movement to the disk end.");
    }
    else
    {
        printf("\n\nConclusion: Both algorithms have equal efficiency.");
    }
    return 0;
}

```

OUTPUT:

```

Output
Enter disk size: 200
Enter number of requests: 8
Enter request queue:
98 183 37 122 14 124 65 67
Enter initial head position: 53
Enter direction (1 = Right, 0 = Left): 1

=====
          DISK SCHEDULING RESULTS
=====
--- C-SCAN ---
Seek Sequence: 53 -> 65 -> 67 -> 98 -> 122 -> 124 -> 183 -> 199 -> 0 -> 14 -> 37
Total Head Movement = 382

--- LOOK ---
Seek Sequence: 53 -> 65 -> 67 -> 98 -> 122 -> 124 -> 183 -> 37 -> 14
Total Head Movement = 299

=====
          EFFICIENCY COMPARISON
=====
C-SCAN Total Head Movement = 382
LOOK    Total Head Movement = 299

Conclusion: LOOK performs better.
Reason: It avoids unnecessary movement to the disk end.

=== Code Execution Successful ===

```

CODING:

```
#include <stdio.h>
#include <stdlib.h>
#define MAX 100
/* Check whether a block already exists in the request queue */
int exists(int arr[], int n, int block)
{
    int i;
    for (i = 0; i < n; i++)
    {
        if (arr[i] == block)
            return 1;
    }
    return 0;
}
/* Contiguous Allocation */
int contiguous(int disk[], int n, int fileSize, int allocated[])
{
    int i, j, count;
    for (i = 0; i <= n - fileSize; i++)
    {
        count = 0;
        for (j = i; j < i + fileSize; j++)
        {
            if (disk[j] == 0)
                count++;
            else
                break;
        }
    }
}
```

```

    }
    if (count == fileSize)
    {
        for (j = 0; j < fileSize; j++)
        {
            disk[i + j] = 1;
            allocated[j] = i + j;
        }
        return 1;
    }
}
return 0;
}

/* Indexed Allocation */
int indexed(int disk[], int n, int fileSize, int allocated[])
{
    int i, count = 0;
    int indexBlock = -1;
    /* One extra block is required for index */
    if (fileSize + 1 > n)
        return 0;
    /* Find index block */
    for (i = 0; i < n; i++)
    {
        if (disk[i] == 0)
        {
            indexBlock = i;
            disk[i] = 1;
            break;
        }
    }
}

```

```

    }
}
if (indexBlock == -1)
    return 0;
/* Allocate data blocks */
for (i = 0; i < n && count < fileSize; i++)
{
    if (disk[i] == 0)
    {
        disk[i] = 1;
        allocated[count] = i;
        count++;
    }
}
if (count < fileSize)
    return 0;
printf("Index Block: %d\n", indexBlock);
return 1;
}
/* FCFS Scheduling */
int FCFS(int requests[], int n, int head)
{
    int i;
    int current = head;
    int total = 0;
    printf("Seek Sequence: %d", head);
    for (i = 0; i < n; i++)
    {
        total += abs(current - requests[i]);
    }
}

```

```

        current = requests[i];
        printf(" -> %d", current);
    }
    printf("\nTotal Seek Time: %d\n", total);
    return total;
}

/* SSTF Scheduling */
int SSTF(int requests[], int n, int head)
{
    int visited[MAX] = {0};
    int i, j;
    int current = head;
    int total = 0;
    int minDistance, index;
    printf("Seek Sequence: %d", head);
    for (i = 0; i < n; i++)
    {
        minDistance = 99999;
        index = -1;
        for (j = 0; j < n; j++)
        {
            if (!visited[j])
            {
                if (abs(current - requests[j]) < minDistance)
                {
                    minDistance = abs(current - requests[j]);
                    index = j;
                }
            }
        }
    }
}

```

```

    }

    visited[index] = 1;
    total += abs(current - requests[index]);
    current = requests[index];
    printf(" -> %d", current);
}

printf("\nTotal Seek Time: %d\n", total);
return total;
}

/* Display allocated blocks */
void displayBlocks(int allocated[], int fileSize)
{
    int i;
    printf("Allocated Blocks: ");
    for (i = 0; i < fileSize; i++)
    {
        printf("%d ", allocated[i]);
    }
    printf("\n");
}

/* Run one combination */
int runCombination(int allocationMethod,
                  int schedulingMethod,
                  int originalDisk[],
                  int diskSize,
                  int fileSize,
                  int inputQueue[],
                  int queueSize,

```

```

        int head)
{
    int disk[MAX];
    int allocated[MAX];
    int requests[MAX];
    int i, n, seekTime;
    int allocationResult;
    /* Copy original disk */
    for (i = 0; i < diskSize; i++)
        disk[i] = originalDisk[i];
    /* Allocate blocks */
    if (allocationMethod == 1)
    {
        allocationResult =
            contiguous(disk, diskSize, fileSize, allocated);
    }
    else
    {
        allocationResult =
            indexed(disk, diskSize, fileSize, allocated);
    }
    if (!allocationResult)
    {
        printf("Allocation failed.\n");
        return -1;
    }
    displayBlocks(allocated, fileSize);
    n = 0;
    for (i = 0; i < queueSize; i++)

```



```

{
    if (!exists(requests, n, inputQueue[i]))
    {
        requests[n] = inputQueue[i];
        n++;
    }
}
for (i = 0; i < fileSize; i++)
{
    if (!exists(requests, n, allocated[i]))
    {
        requests[n] = allocated[i];
        n++;
    }
}
/* Perform scheduling */
if (schedulingMethod == 1)
{
    seekTime = FCFS(requests, n, head);
}
else
{
    seekTime = SSTF(requests, n, head);
}
return seekTime;
}
int main()
{
    int disk[MAX];

```

```

int requestQueue[MAX];
int n, diskSize, fileSize;
int head;
int i;
int results[4];
int best = -1;
int bestTime = 99999;

printf("=====\n");
printf(" FILE ALLOCATION + DISK SCHEDULING SYSTEM\n");
printf("=====\n");

printf("\nEnter total number of disk blocks: ");
scanf("%d", &diskSize);
if (diskSize <= 0 || diskSize > MAX)
{
    printf("Invalid disk size.\n");
    return 0;
}

printf("Enter initial disk bitmap\n");
printf("(0 = Free, 1 = Allocated):\n");
for (i = 0; i < diskSize; i++)
{
    scanf("%d", &disk[i]);
    if (disk[i] != 0 && disk[i] != 1)
    {
        printf("Invalid bitmap value.\n");
        return 0;
    }
}

printf("\nEnter file size (number of blocks required): ");

```

```

scanf("%d", &fileSize);
if (fileSize <= 0 || fileSize > diskSize)
{
    printf("Invalid file size.\n");
    return 0;
}
printf("\nEnter number of disk requests: ");
scanf("%d", &n);
if (n < 0 || n > MAX)
{
    printf("Invalid number of requests.\n");
    return 0;
}
printf("Enter disk request queue:\n");
for (i = 0; i < n; i++)
{
    scanf("%d", &requestQueue[i]);

    if (requestQueue[i] < 0 ||
        requestQueue[i] >= diskSize)
    {
        printf("Invalid disk request: %d\n",
            requestQueue[i]);
        return 0;
    }
}
printf("\nEnter initial head position: ");
scanf("%d", &head);

```

```

        if (head < 0 || head >= diskSize)
        {
            printf("Invalid head position.\n");
            return 0;
        }
printf("\n=====");
        printf("\n1. CONTIGUOUS + FCFS");
printf("\n=====\\n"
);
        results[0] = runCombination(
            1, 1, disk, diskSize, fileSize,
            requestQueue, n, head);
        if (results[0] != -1 && results[0] < bestTime)
        {
            bestTime = results[0];
            best = 0;
        }
printf("\n=====");
        printf("\n2. CONTIGUOUS + SSTF");
printf("\n=====\\n");
        results[1] = runCombination(
            1, 2, disk, diskSize, fileSize,
            requestQueue, n, head);
        if (results[1] != -1 && results[1] < bestTime)
        {
            bestTime = results[1];
            best = 1;
        }

```

```

printf("\n=====");
    printf("\n3. INDEXED + FCFS");
printf("\n=====\\n");
    results[2] = runCombination(
        2, 1, disk, diskSize, fileSize,
        requestQueue, n, head);
    if (results[2] != -1 && results[2] < bestTime)
    {
        bestTime = results[2];
        best = 2;
    }
printf("\n=====");
    printf("\n4. INDEXED + SSTF");
printf("\n=====\\n"
);
    results[3] = runCombination(
        2, 2, disk, diskSize, fileSize,
        requestQueue, n, head);
    if (results[3] != -1 && results[3] < bestTime)
    {
        bestTime = results[3];
        best = 3;
    }

printf("\n\\n=====\\n"
);
    printf("\n      PERFORMANCE COMPARISON");

```

```

printf("\n===== \n"
);
printf("\nContiguous + FCFS : ");
if (results[0] == -1)
    printf("Allocation Failed\n");
else
    printf("%d seek time\n", results[0]);
printf("Contiguous + SSTF : ");
if (results[1] == -1)
    printf("Allocation Failed\n");
else
    printf("%d seek time\n", results[1]);
printf("Indexed + FCFS : ");
if (results[2] == -1)
    printf("Allocation Failed\n");
else
    printf("%d seek time\n", results[2]);
printf("Indexed + SSTF : ");
if (results[3] == -1)
    printf("Allocation Failed\n");
else
    printf("%d seek time\n", results[3]);
if (best != -1)
{
    printf("\n-----");
    printf("\nBEST COMBINATION\n");
    printf("-----\n");
    if (best == 0)

```

```

        printf("Contiguous Allocation + FCFS\n");
    else if (best == 1)
        printf("Contiguous Allocation + SSTF\n");
    else if (best == 2)
        printf("Indexed Allocation + FCFS\n");
    else
        printf("Indexed Allocation + SSTF\n");
    printf("Minimum Total Seek Time = %d\n", bestTime);
}
else
{
    printf("\nNo combination was successful.\n");
    printf("Reason: File allocation failed for all methods.\n");
}
return 0;
}

```

OUTPUT:

```

Output
=====
FILE ALLOCATION + DISK SCHEDULING SYSTEM
=====

Enter total number of disk blocks: 20
Enter initial disk bitmap
(0 = Free, 1 = Allocated):
1 0 0 1 0 0 0 1 0 0 1 0 0 0 1 0 0 0 1 0

Enter file size (number of blocks required): 4

Enter number of disk requests: 5
Enter disk request queue:
12 5 18 2 15

Enter initial head position: 10

```

```
Output
12 5 18 2 15

Enter initial head position: 10

=====
1. CONTIGUOUS + FCFS
=====
Allocation failed.

=====
2. CONTIGUOUS + SSTF
=====
Allocation failed.

=====
3. INDEXED + FCFS
=====
Index Block: 1
Allocated Blocks: 2 4 5 6
Seek Sequence: 10 -> 12 -> 5 -> 18 -> 2 -> 15 -> 4 -> 6
Total Seek Time: 64

=====
4. INDEXED + SSTF
=====
Index Block: 1
Allocated Blocks: 2 4 5 6
Seek Sequence: 10 -> 12 -> 15 -> 18 -> 6 -> 5 -> 4 -> 2
Total Seek Time: 24
```

```
=====
4. INDEXED + SSTF
=====
Index Block: 1
Allocated Blocks: 2 4 5 6
Seek Sequence: 10 -> 12 -> 15 -> 18 -> 6 -> 5 -> 4 -> 2
Total Seek Time: 24

=====
PERFORMANCE COMPARISON
=====

Contiguous + FCFS : Allocation Failed
Contiguous + SSTF : Allocation Failed
Indexed + FCFS    : 64 seek time
Indexed + SSTF    : 24 seek time

=====
BEST COMBINATION
=====

Indexed Allocation + SSTF
Minimum Total Seek Time = 24

=== Code Execution Successful ===
```